

universidad de león Online

Universidad de León ▪ Máster Universitario en Ciberseguridad

# Ingeniería del software seguro en la era de la IA generativa

## Diseño y Programación Seguras

---

## Principios clásicos y diseño seguro

Francisco Javier Rodríguez Lera ▪ Camino Fernández Llamas



Instituto de Investigación e Innovación en Ingeniería

Escuela de Ingenierías

universidad de León

# Objetivos de aprendizaje

Al terminar el capítulo, el estudiante será capaz de:

- **Distinguir** heurística, observación y ley; usar las máximas como herramientas *falibles*.
- **Analizar** KISS, DRY, YAGNI, Kernighan, Brooks, Pareto, Conway y Dunbar: qué asumen y cómo fallan.
- **Aplicar** los principios de Saltzer y Schroeder al diseño seguro.
- **Razonar** con defensa en profundidad y diseño por contrato.
- **Decidir**, en proyectos asistidos por IA, qué cambia y qué permanece.

# Heurística no es ley

La ingeniería acumula máximas que se repiten como leyes físicas. Cautela terminológica:

**Heurística** regla de decisión que suele dar buenos resultados en un rango de contextos; admite excepciones.

**Observación** patrón empírico o sociotécnico, no demostrado como teorema.

Confundir ambas lleva a dos errores simétricos: descartar una heurística porque «tiene excepciones», o invocarla como argumento de autoridad incontestable.

# Naturaleza y ámbito de los principios

| Principio | Naturaleza | Ámbito | Estatus |
|---|---|---|---|
| KISS | Heurística de diseño | Complejidad estructural | Propuesta docente |
| DRY | Heurística de diseño | Representación del conocimiento | Propuesta docente |
| YAGNI | Heurística de alcance | Gestión de requisitos | Propuesta docente |
| Kernighan | Observación (aforismo) | Comprensión del código | Hipótesis |
| Brooks | Observación empírica | Gestión de proyectos | Evidencia parcial |
| Pareto | Heurística estadística | Priorización de esfuerzo | Hipótesis |
| Conway | Observación sociotécnica | Arquitectura y organización | Hipótesis |
| Dunbar | Hipótesis científica | Tamaño de redes sociales | Hipótesis discutida |

«Hipótesis» no significa «falso», sino «no demostrado». «Propuesta docente» señala una decisión pedagógica, no un resultado científico.

## DISCUSIÓN ¿Siguen vigentes las leyes elementales?

- Si la IA reduce el coste de escribir código, ¿qué ocurre con estas heurísticas?
- ¿Se vuelven obsoletas o cambia su *economía*?

# KISS: mantenerlo simple

Prefiere la solución más sencilla que satisfaga el requisito, antes que una más general, abstracta o «lista» de lo necesario.

- **Naturaleza**: heurística de diseño; la complejidad puede ser intrínseca.
- **Problema**: la *complejidad accidental* encarece el mantenimiento y oculta defectos.
- **Supuestos**: el coste dominante es *comprender y modificar*, no escribir; la complejidad innecesaria es evitable.

# KISS con IA

## Cambios con IA

- La IA reduce el coste de *escribir*: rebaja el coste aparente de *añadir* complejidad.
- Riesgo invertido: es fácil generar más código del necesario.
- La simplicidad pasa de restricción económica a *disciplina deliberada*.

## Riesgos de seguridad

- Sobreingeniería: más superficie de ataque y dependencias.
- Módulos «genéricos» con rutas que nadie audita.
- Complejidad que favorece el *fallo silencioso*.

Invariante: toda línea es una línea que alguien deberá comprender, revisar y asegurar. Menos complejidad puede reducir fallos, pero el número de líneas no mide por sí solo la seguridad.

## DISCUSIÓN ¿Puede KISS reducir la seguridad?

- ¿En qué escenario aplicar KISS produciría una solución *menos* segura que una alternativa más compleja?

# DRY: no te repitas

Cada pieza de *conocimiento* debe tener una representación única, inequívoca y autoritativa. No prohíbe duplicar código, sino duplicar *decisiones*.

- **Problema**: copias que divergen con el tiempo; se corrige una y se olvidan las demás.
- **Supuestos**: fuente de verdad identificable; el coste de abstraer es menor que el de mantener copias; la abstracción no acopla peor.

# DRY y controles de seguridad

- Los asistentes *replican patrones* con facilidad: agravan el riesgo de duplicar lógica de validación.
- También pueden proponer abstracciones prematuras que ocultan el flujo.
- **Invariante**: las reglas de validación y seguridad residen en un *único* punto autoritativo.
- Si la canonicalización de rutas o el escapado de fórmulas se duplican, divergen, y la copia más débil determina la seguridad del conjunto.

**Duplicación de controles** = validación inconsistente. Una abstracción mal diseñada puede concentrar el fallo: un único componente defectuoso afecta a todos los llamantes.

# YAGNI: no lo vas a necesitar

No implementes funcionalidad hasta que exista una necesidad real; no la implementes por especulación sobre requisitos futuros.

- **Naturaleza**: heurística de gestión de alcance (ágil, XP).
- **Problema**: la funcionalidad especulativa se construye, documenta, prueba y mantiene antes de que nadie la necesite.
- **Invariante**: cada funcionalidad debe trazarse a un requisito explícito.

# YAGNI y seguridad

- La IA genera funcionalidad «de reserva» con poco esfuerzo aparente: erosiona la disciplina de YAGNI.
- La funcionalidad no solicitada suele ser la *menos revisada*: deserializadores «por compatibilidad», modos de depuración «por si acaso».
- En el caso transversal, aceptar formatos adicionales sin requisito equivale a incorporar analizadores nuevos con su propia superficie de ataque.

YAGNI **no** autoriza a ignorar requisitos de seguridad: los límites de tamaño, la canonicalización de rutas y el escapado de salidas son *condiciones del requisito*, no funcionalidad especulativa.

# Ley de Kernighan

> Depurar es el doble de difícil que escribir código; si escribes el código tan hábilmente como puedes, por definición no eres lo bastante hábil para depurarlo.

- **Naturaleza**: observación expresada como aforismo; **Hipótesis**.
- **Problema**: el código «demasiado listo» es difícil de entender, depurar y auditar.

# Kernighan con IA

- La asimetría se agrava: el «escritor» es un modelo que no depura y no comparte contexto con el revisor humano.
- El desarrollador revisa código que no escribió y que puede ser más «listo» de lo que él habría producido.
- La comprensión deja de ser consecuencia de haber escrito: pasa a ser una *tarea explícita*.

El código ofuscado o compacto oculta fallos: un desbordamiento, una comprobación ausente o una rama inalcanzable pueden pasar inadvertidos en una revisión superficial.

# Ley de Brooks

Añadir personal a un proyecto atrasado lo retrasa aún más. La complejidad se divide en *esencial* (inherente al problema) y *accidental* (de herramientas y proceso); ninguna tecnología elimina la primera.

- **Naturaleza**: observación empírica; **Evidencia parcial**.
- **Problema**: los costes de comunicación e integración superan la capacidad añadida.

# Brooks con IA: no es bala de plata

- La IA reduce la complejidad *accidental* (andamiaje, código repetitivo), pero **no** la *esencial* del dominio.
- Introduce costes nuevos: verificación, revisión, gestión de la confianza y dependencia de proveedores.
- La complejidad esencial del caso -validar entradas, manejar formatos, asegurar salidas- permanece aunque se genere más código.

Si se confía en la IA como solución universal, la revisión humana se relaja donde más se necesita.

# Principio de Pareto

Una fracción reducida de las causas produce la mayor parte de los efectos; en su forma popular, «el 80 % de los resultados proviene del 20 % de las causas».

- **Naturaleza**: heurística estadística; el 80/20 es ilustrativo, no una constante universal; **Hipótesis** aplicada al software.
- **Problema**: ¿dónde concentrar el esfuerzo con recursos limitados? En la *minoría crítica*.

# Pareto con IA

- La IA acelera el «80» trivial (código repetitivo, andamiaje) pero el «20» crítico (casos límite, condiciones de carrera, entradas hostiles) sigue requiriendo análisis humano.
- Riesgo: el volumen generado da una falsa sensación de progreso mientras el subconjunto crítico queda sin atender.
- Las rutas de error poco frecuentes suelen ser las más vulnerables.

No confundir el volumen de código con el valor entregado. El esfuerzo de seguridad debe dirigirse al subconjunto que concentra el riesgo.

# Ley de Conway

Las organizaciones tienden a producir sistemas cuyo diseño copia su estructura de comunicación: las fronteras entre módulos reflejan las fronteras entre equipos.

- **Naturaleza**: observación sociotécnica; **Hipótesis**.
- **Invariante**: la arquitectura de seguridad debe reflejar las fronteras de *confianza* y *responsabilidad*, no solo las organizativas.

# Conway con IA

- La IA introduce un nuevo «participante»: asistentes y agentes que producen y a veces ejecutan código.
- ¿Cómo se refleja una organización que incluye herramientas automatizadas? Es una pregunta abierta (**Hipótesis**).
- Si las fronteras de confianza no coinciden con las organizativas, los controles se colocan en el sitio equivocado.
- La responsabilidad de revisar código generado puede quedar sin dueño claro.

# Número de Dunbar

Se ha propuesto un límite cognitivo al número de relaciones sociales estables, en torno a un centenar y medio. **Hipótesis científica discutida**.

- Derivada de correlaciones entre neocórtex y tamaño de grupo en primates.
- **No debe extrapolarse** sin reservas a equipos de ingeniería.
- Su valor pedagógico es *negativo*: ejemplo de hipótesis atractiva pero insegura.

No debe usarse como norma de diseño organizativo ni como justificación del tamaño de un equipo de revisión.

# Dunbar: una lección metodológica

- Una hipótesis popular no se convierte en evidencia por repetirse.
- El mismo criterio que exige verificar una CWE [1] o una edición de un estándar [2] se aplica a las máximas de gestión: preguntar por las condiciones de validez.
- Para dimensionar un equipo: usar carga de trabajo, competencias y trazabilidad, no una constante biológica dudosa.

# Principios de diseño seguro: Saltzer y Schroeder

Conjunto formulado en 1975; referencia histórica *de ingeniería*, no un catálogo de obligaciones legales. Su vigencia se analiza frente al modelo de amenazas y la plataforma actuales.

| **Principio** | **Idea central** |
|---|---|
| Mínimo privilegio | Solo los permisos necesarios, durante el tiempo necesario. |
| Valores seguros por defecto | Lo no autorizado se deniega por defecto. |
| Economía de mecanismos | Base de confianza pequeña y revisable. |
| Mediación completa | Todo acceso a objeto protegido se comprueba. |
| Separación de privilegios | Operación sensible: dos condiciones independientes. |
| Mínimo mecanismo común | Reducir mecanismos compartidos entre niveles de confianza. |
| Diseño abierto | La seguridad no depende del secreto del diseño. |
| Aceptabilidad psicológica | Controles comprensibles y utilizables. |

# Mínimo privilegio y valores por defecto

## Mínimo privilegio

- Una vulnerabilidad local no debe escalar a compromiso general.
- **Con IA**: un agente que solo propone cambios no necesita ejecutar comandos, publicar paquetes ni acceder a secretos.
- Separar generación, revisión y ejecución autorizada.
- *Límite*: privilegios insuficientes pueden impedir una función.

## Valores seguros por defecto

- Permitir exige autorización explícita; lo demás se deniega.
- **Con IA**: ante un parámetro omitido, no inferir que todo destino es admisible.
- *Contraejemplo*: exportar a cualquier ruta si falta configuración = autorización implícita.

**Ejemplo**: el conversor recibe lectura del directorio de entrada y escritura solo del directorio de salida, no del árbol completo del proyecto.

# Economía, mediación completa y separación de privilegios

- **Economía de mecanismos**: base de confianza pequeña y revisable. Una criptografía casera de veinte líneas *no* es preferible a una biblioteca consolidada solo por ser más corta.
- **Mediación completa**: todo acceso se somete a comprobación. Validar una ruta una vez *no* resuelve carreras TOCTOU.
- **Separación de privilegios**: una operación sensible puede requerir dos condiciones independientes. Publicar un artefacto puede exigir pruebas de seguridad ejecutadas *y* aprobación humana.

No confundir separación de privilegios con separación de responsabilidades del diseño OO: se complementan, pero resuelven problemas distintos.

# Mínimo mecanismo común, diseño abierto y aceptabilidad

- **Mínimo mecanismo común**: un agente con código no confiable no debería compartir credenciales, temporales sensibles ni estado con producción.
- **Diseño abierto**: la seguridad no depende del secreto del diseño; los secretos operativos (claves) sí se protegen. La disponibilidad del código no es prueba de seguridad (Kerckhoffs).
- **Aceptabilidad psicológica**: controles comprensibles. Un flujo con decenas de solicitudes indistinguibles favorece la aceptación mecánica.

Los principios de Saltzer y Schroeder *no* son una lista de verificaciones que certifique la seguridad: hacen explícitas decisiones para discutirlas y someterlas a pruebas.

# Defensa en profundidad

Combina controles con *modos de fallo diferentes*: validación, privilegios mínimos, aislamiento, revisión, análisis estático, pruebas dinámicas, monitorización y recuperación.

- No equivale a multiplicar controles redundantes sin justificación.
- Dos verificadores que comparten el mismo supuesto erróneo pueden fallar simultáneamente.
- Con IA: el código generado se trata como *no verificado*, aunque compile y supere pruebas de ejemplo.

# Defensa en profundidad en el caso

- El conversor valida el esquema CSV, limita tamaño y recursos, restringe rutas de salida y comprueba que el documento no contiene fórmulas interpretables.

**Contraejemplo**: un *pipeline* con diez pruebas positivas *no* demuestra que una entrada maliciosa sea rechazada. Se necesitan pruebas negativas y un modelo explícito de amenazas.

Las acciones del agente también necesitan controles independientes de los del código que produce.

# Diseño por contrato, invariantes y responsabilidad

Formula las obligaciones de un componente mediante precondiciones, poscondiciones e invariantes (lógica de Hoare, capítulo 3).

**Precondición.** El CSV satisface un límite de tamaño, un esquema declarado y una codificación aceptada.

**Poscondición.** La salida representa los registros admitidos sin interpretar contenido de usuario como fórmulas.

**Invariante.** Ninguna operación escribe fuera del directorio autorizado, incluso ante errores y ejecuciones concurrentes.

Estas propiedades *no* se obtienen automáticamente al redactar un contrato: deben implementarse y verificarse bajo supuestos explícitos. Una poscondición informal no sustituye una prueba formal.

# Otros principios con interpretación contextual

- **Menor sorpresa**: interfaz coherente con expectativas y contrato. En código IA, revisar opciones por defecto y funcionalidades añadidas. *Predecible* $\neq$ *seguro*.
- **Ley de Postel**: «conservador al enviar, liberal al recibir» facilita interoperabilidad, pero la aceptación permisiva crea discrepancias entre analizadores. En límites de confianza, mejor formatos inequívocos.
- **Responsabilidad única**: separa lectura, validación, transformación y escritura; no sustituye las políticas de acceso.
- **Sustitución de Liskov**: sustituir un componente no es seguro si debilita poscondiciones o cambia garantías de confidencialidad y acceso.

# Matriz de decisión para proyectos asistidos por IA

| Principio | Pregunta verificable para el revisor |
|---|---|
| Mínimo privilegio | ¿Qué recursos puede leer, modificar o ejecutar el agente? |
| Valores por defecto | ¿Qué ocurre si falta un permiso o una configuración? |
| Mediación completa | ¿Se verifica cada operación y se evitan carreras TOCTOU? |
| Separación de privilegios | ¿Qué acciones requieren aprobación o condiciones independientes? |
| Economía de mecanismos | ¿Es revisable la base de confianza? |
| Defensa en profundidad | ¿Qué control sigue operando si otro falla? |
| Diseño por contrato | ¿Qué precondiciones, poscondiciones e invariantes se comprueban? |
| Menor sorpresa | ¿Existen efectos secundarios o capacidades no solicitadas? |

# Caso transversal: ¿cuántos formatos soportar?

Decisión concreta: ¿cuántos formatos debe soportar el programa? Separar tres dimensiones:

| **Dimensión** | **Solo CSV** | **CSV + PDF + XLSX** |
|---|---|---|
| Dependencias | Biblioteca estándar | Varias externas; versiones y licencias |
| Superficie | Analizador sencillo | Analizadores complejos; formatos comprimidos |
| Validación | Comillas, separadores | Además: fórmulas, XML, comprimidos |
| Mantenimiento | Bajo | Alto; compatibilidad por formato |
| Riesgo principal | Inyección en salida CSV | Fórmulas, deserialización, *path traversal* |

CASO TRANSVERSAL ▪ CSV–PDF–XLSX

# Inyección de fórmulas: causa y mitigación

- Un campo que empieza por =, +, − o @ puede ejecutarse como fórmula al abrirse en una hoja de cálculo.
- Es **CWE-1236** [1]; *no* debe atribuirse sin más al OWASP Top 10 para aplicaciones web.
- La neutralización de prefijos es una medida *contextual*: considerar también TAB, CR y espacios iniciales; depende del programa que abra el fichero.

# Inyección de fórmulas (CWE-1236)

```python
# Neutraliza la interpretacion de formulas en hojas de calculo.
# La correccion es CONTEXTUAL: no demuestra seguridad frente a todas las entradas.
PELIGROSOS = ("=", "+", "-", "@")

def escapar_celda(valor: str) -> str:
    if valor.startswith(PELIGROSOS):
        return "'" + valor          # mitigacion contextual: verificar en el destino
    return valor
```

Listing 1: Escapado de fórmulas (ejemplo didáctico, corrección contextual).

# Cautela sobre la mitigación

El control del listado debe residir en un *único* punto autoritativo (DRY) y aplicarse a toda celda exportada, no solo a algunas. Su corrección es *contextual*: depende del formato de destino y de la versión del programa que lo abra.

Ningún conjunto finito de pruebas sobre estos formatos demuestra que el programa sea seguro. Superar los casos elegidos solo indica que *esos* casos pasan. Un ejemplo «corregido» es correcto de forma contextual, nunca «universalmente seguro».

# Síntesis: efecto de la IA sobre cada principio

| Principio | Lo que abarata | Lo que encarece o exige |
|---|---|---|
| KISS | Escribir y añadir código | Imponer simplicidad deliberadamente |
| DRY | Reproducir patrones | Evitar duplicación de controles |
| YAGNI | Generar funcionalidad de reserva | Justificar cada funcionalidad por requisito |
| Kernighan | Producir código «listo» | Comprender código ajeno antes de revisarlo |
| Brooks | Reducir complejidad accidental | Gestionar verificación y confianza |
| Pareto | Producir el «80» trivial | Atender el «20» crítico |
| Conway | Generar módulos | Definir fronteras de confianza y responsabilidad |
| Dunbar | - (sin efecto claro) | No invocar hipótesis no verificadas |

La IA abarata lo que se *escribe*, sin reducir proporcionalmente el esfuerzo de *comprender, especificar y verificar*. El punto de aplicación se desplaza del escribir al decidir y validar.

# Actividad

1. **Clasificación epistémica**: para cada principio, ¿ley, observación o heurística? Justifique; ¿cuándo cambia la decisión de diseño?
2. **Contraejemplo a KISS**: un escenario donde lo simple sea *menos* seguro.
3. **DRY**: tres reglas de validación del caso que deberían tener representación única; qué riesgo aparece si se duplican.
4. **YAGNI**: un compañero propone PDF y Excel «porque la IA lo hace en dos minutos». Distinga coste de escribir de coste de mantener y asegurar.
5. **Dunbar**: dos supuestos no verificados de «los equipos de revisión no deben superar 150 personas»; proponga un criterio alternativo verificable.

## DISCUSIÓN ¿Qué principio se invierte con la IA?

- ¿En qué caso el abaratamiento de la escritura convierte una buena heurística en una mala práctica?
- ¿Qué disciplina debe imponerse deliberadamente para evitarlo?

# Referencias I




MITRE Corporation.

Common weakness enumeration (cwe).

Catálogo de referencia; los identificadores deben justificarse por caso.



National Institute of Standards and Technology.

Secure software development framework (ssdf) version 1.1: Recommendations for mitigating the risk of software vulnerabilities.

Technical Report SP 800-218, NIST, 2022.

Norma / marco de referencia.